



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Παράλληλα Συστήματα
Εργασία 3(2025-2026)

Ονοματεπώνυμο : Ζουμπουρλής Θοδωρής

Αριθμός Μητρώου : 1115202200267

Ονοματεπώνυμο : Τσάκωνας Βασίλης

Αριθμός Μητρώου : 1115202200245

Περιεχόμενα

1	Πληροφορίες για υπολογιστικό σύστημα	2
2	3.1	2
3	3.2	7

1 Πληροφορίες για υπολογιστικό σύστημα

Όνομα υπολογιστικού συστήματος: linux02.di.uoa.gr

Μοντέλο επεξεργαστή: Intel(R) Core(TM) i7-6700 CPU @ 3.40GHz

Ο επεξεργαστής περιέχει 4 φυσικούς πυρήνες, και κάθε πυρήνας υποστηρίζει 2 νήματα μέσω Hyper-Threading. Επομένως, διαθέτει 8 λογικούς επεξεργαστές που μπορούν να εκτελούν 8 νήματα ταυτόχρονα.

Έκδοση μεταγλωττιστή: gcc 9.4.0 (Ubuntu 9.4.0-1ubuntu1~20.04.2)

Έκδοση μεταγλωττιστή: gcc 11.4.0 (Ubuntu 11.4.0-1ubuntu1 ~22.04.2)

Στα πλαίσια της άσκησης χρησιμοποιήθηκε το linux lab της σχολής σύμφωνα με τις οδηγίες που μας έχουν δοθεί.

Επισημάνσεις

Στους επιμέρους φακέλους μπορεί κάποιος να βρει τα scripts που χρησιμοποιήθηκαν για τη συλλογή, παραγωγή και ανάλυση των αποτελεσμάτων. Το κάθε script το τρέξαμε 5 φορές και έπειτα συλλέξαμε τα αποτελέσματα και παρουσιάζουμε τον μέσο όρο τους. Επιπλέον, υπάρχουν και τα python scripts που χρησιμοποιήθηκαν για τη παραγωγή των graphs.

2 3.1

Πρόβλημα και υλοποίηση

Το πρόβλημα που καλούμαστε να υλοποιήσουμε είναι ο πολλαπλασιασμός πολυωνύμων με MPI και σειριακά. Αρχικά υπολογίζουμε το αποτέλεσμα με έναν σειριακό αλγόριθμο για το process 0 και στη συνέχεια ανάλογα με τον αριθμό των processes που έχουμε δώσει ως είσοδο υπολογίζουμε πάλι το ίδιο πολυώνυμο με έναν παράλληλο αλγόριθμο μέσω λήψης και αποστολής μηνυμάτων, χρησιμοποιώντας της εντολές MPI_Bcast, MPI_Scatterv, MPI_Reduce.

Πιο συγκεκριμένα:

MPI_Bcast: Χρησιμοποιείται για την αποστολή ολόκληρου του δεύτερου πολυωνύμου (P2) σε όλες τις διεργασίες, καθώς κάθε τμήμα του πρώτου πολυωνύμου πρέπει να πολλαπλασιαστεί με όλο το P2.

MPI_Scatterv: Επιλέχθηκε η έκδοση "v" (variable) αντί της απλής MPI_Scatter για να υπάρχει ευελιξία στον διαμοιρασμό των συντελεστών του P1, ειδικά αν ο βαθμός του πολυωνύμου δεν είναι τέλεια διαιρετός με τον αριθμό των διεργασιών.

MPI_Reduce: Χρησιμοποιείται για την τελική άθροιση των επιμέρους αποτελεσμάτων στη διεργασία 0, εξασφαλίζοντας ότι το τελικό πολυώνυμο θα συγκεντρωθεί σωστά και αποδοτικά.

Αποτελέσματα

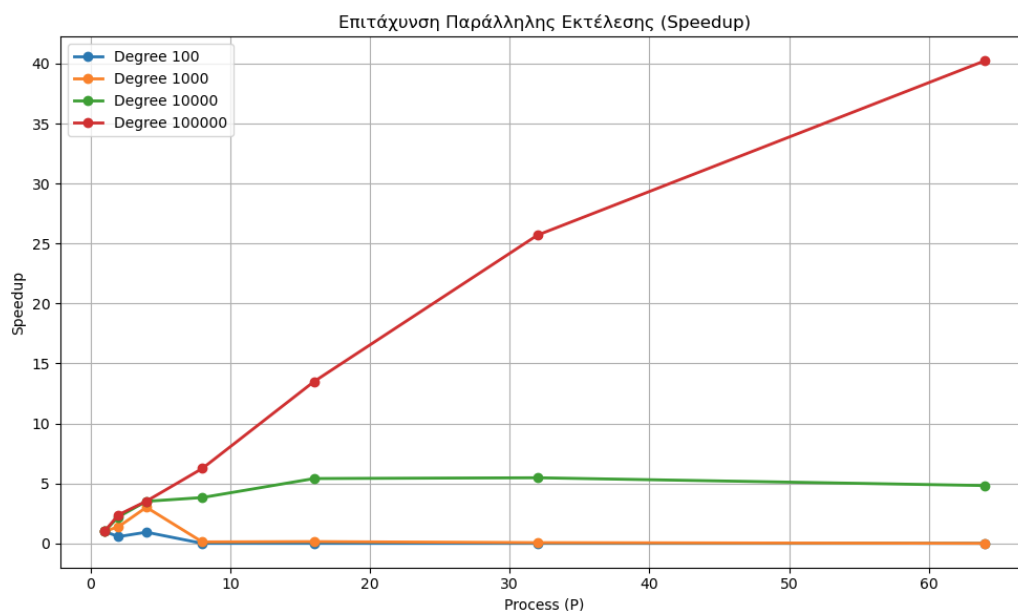
Polynomial Degree	100					
Number of processes	1	2	4	8	16	32
Communication Time from process 0	-	0.0000175	0.0000193	0.0049325	0.015386	0.0182
Calculation Time	-	0.0000255	0.0000133	0.0000063	0.0000035	0.00000225
Receive Time	-	0.0000365	0.0000153	0.012025	0.0216	0.0286
Total Time	0.00004475	0.0000798	0.000048	0.0169	0.037	0.0468
Speedup	1.00	0.561	0.932	0.0026	0.0012	0.00096
Efficiency	1.00	0.281	0.233	0.00033	0.00008	0.00003

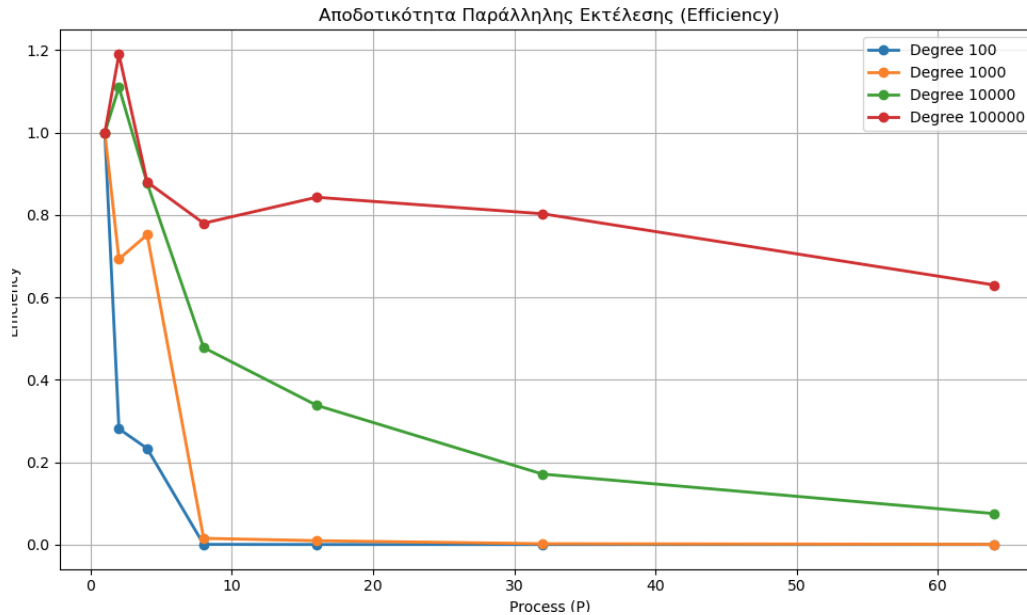
Polynomial Degree	1000					
Number of processes	1	2	4	8	16	32
Communication Time from process 0	-	0.0000245	0.00003025	0.00722	0.0108	0.0195
Calculation Time	-	0.002686	0.00114	0.000625	0.0003005	0.000137
Receive Time	-	0.0001128	0.0001245	0.02412	0.0152	0.04342
Total Time	0.00391	0.002823	0.0013	0.03197	0.02635	0.063
Speedup	1.00	1.385	3.007	0.122	0.148	0.06
Efficiency	1.00	0.693	0.752	0.015	0.0093	0.0019

Polynomial Degree	10000						
Number of processes	1	2	4	8	16	32	64
Communication Time from process 0	-	0.0000778	0.0001078	0.02625	0.02276	0.033	0.0425
Calculation Time	-	0.2099	0.1211	0.0633	0.03115	0.01358	0.00415
Receive Time	-	0.002122	0.01230	0.03292	0.03265	0.03906	0.0508
Total Time	0.468	0.2121	0.1335	0.1225	0.0866	0.0856	0.097
Speedup	1.00	2.21	3.506	3.82	5.404	5.47	4.82
Efficiency	1.00	1.11	0.877	0.478	0.338	0.171	0.075

Polynomial Degree	100000						
Number of processes	1	2	4	8	16	32	64
Communication Time from process 0	-	0.000421	0.000934	0.0232	0.0287	0.0295	0.0739
Calculation Time	-	15.903	11.276	6.170	2.7584	1.001	0.765
Receive Time	-	3.018	1.541	1.0306	0.5516	0.723	0.281
Total Time	45.06	18.92	12.81	7.22	3.34	1.753	1.12
Speedup	1.00	2.38	3.52	6.241	13.491	25.705	40.23
Efficiency	1.00	1.19	0.88	0.78	0.843	0.803	0.63

Γραφήματα





Στα παραπάνω γραφήματα παρουσιάζονται η απόδοση και το speedup. Για μεγάλους βαθμούς πολυωνύμων πειραματιστήκαμε χρησιμοποιώντας 64 διεργασίες στους μικρότερους βαθμούς σταματήσαμε στις 32 καθώς η απόδοση ήταν ήδη πολύ χαμηλή.

Ανάλυση Αποτελεσμάτων

Για μικρούς βαθμούς πολυωνύμων (π.χ. 100 και 1000), η παράλληλη υλοποίηση με MPI δεν παρουσιάζει βελτίωση. Το speedup είναι μικρό η απόδοση μειώνεται έντονα, φτάνοντας σε πολύ χαμηλές τιμές καθώς αυξάνεται ο αριθμός των διεργασιών. Αυτό οφείλεται στο γεγονός ότι το communication overhead (MPI_Bcast, MPI_Scatterv, MPI_Reduce) είναι μεγαλύτερο από τον καθαρό χρόνο υπολογισμού.

Όταν ο βαθμός πολυωνύμου είναι 10.000, παρατηρείται σημαντική επιτάχυνση. Το speedup αυξάνεται μέχρι περίπου 5 φορές για μεγάλο αριθμό διεργασιών, όμως η απόδοση μειώνεται αρκετά όταν ξεπερνάμε τις 8+ διεργασίες. Αυτό δείχνει ότι, παρότι υπάρχει κέρδος χρόνου, η αξιοποίηση των πόρων δεν είναι ιδανική.

Για πολύ μεγάλους βαθμούς πολυωνύμων (π.χ. 100.000), το speedup αυξάνεται σημαντικά και φτάνει πάνω από 40 φορές για μεγάλο αριθμό διεργασιών. Παρ' όλα αυτά, η απόδοση μειώνεται δραστηρικά και συνεχίζει να πέφτει όσο αυξάνονται οι διεργασίες. Μετά από ένα σημείο (π.χ. πάνω από 32 διεργασίες), η περαιτέρω αύξηση των κόμβων δεν έχει όφελος, καθώς το επιπλέον κέρδος χρόνου είναι μικρό σε σχέση με το αυξημένο κόστος επικοινωνίας και το κακό load balancing.

Συνολικά, τα αποτελέσματα δείχνουν ότι, αν και η MPI υλοποίηση επιτυγχάνει υψηλό speedup για μεγάλα μεγέθη προβλήματος, η απόδοση μειώνεται σημαντικά, γεγονός που υποδηλώνει ότι η παραλληλοποίηση έχει πρακτικό όριο και η χρήση υπερβολικά πολλών διεργασιών δεν οδηγεί σε αντίστοιχα οφέλη.

Οπότε η καλύτερη περίπτωση για κάθε βαθμό πολυωνύμου, ώστε να έχουμε load-balance είναι η εξής:

- Βαθμός πολυωνύμου 100, 1000: 2-4 διεργασίες
- Βαθμός πολυωνύμου 10000: 8-16 διεργασίες
- Βαθμός πολυωνύμου 100000: 32 διεργασίες

Τα παρακάτω αποτελέσματα αφορούν την εκτέλεση μόνο σε έναν κόμβο χρησιμοποιώντας MPI:

Table Size	100				
Number of processes	1	2	4	8	16
Communication Time from process 0	-	0.0000045	0.00000475	0.000009	0.000136
Calculation Time	-	0.00000625	0.00000325	0.00000175	0.000001
Receive Time	-	0.0000025	0.00000325	0.00444	0.0149
Total Time	0.0000125	0.0000133	0.000011	0.00445	0.015
Speedup	1.00	0.943	1.136	0.003	0.001
Efficiency	1.00	0.472	0.284	0.000	0.000

Table Size	1000				
Number of processes	1	2	4	8	16
Communication Time from process 0	-	0.000006	0.000008	0.000009	0.000137
Calculation Time	-	0.000587	0.000297	0.000155	0.000075
Receive Time	-	0.000013	0.0109	0.000038	0.0175
Total Time	0.00119	0.00061	0.0112	0.000202	0.0177
Speedup	1.00	1.962	0.106	5.886	0.067
Efficiency	1.00	0.981	0.026	0.736	0.004

Table Size	10000				
Number of processes	1	2	4	8	16
Communication Time from process 0	-	0.00001925	0.0000265	0.00004	0.000172
Calculation Time	-	0.0575	0.0292	0.0172	0.0136
Receive Time	-	0.00105	0.000336	0.000915	0.0158
Total Time	0.118	0.0586	0.02956	0.0181	0.0295
Speedup	1.00	2.014	3.994	6.511	4.002
Efficiency	1.00	1.007	0.998	0.814	0.250

Table Size	100000				
Number of processes	1	2	4	8	16
Communication Time from process 0	-	0.000112	0.000224	0.000316	0.0218
Calculation Time	-	5.812	2.931	1.709	1.532
Receive Time	-	0.0002798	0.02104	0.127	0.24
Total Time	11.659	5.812	2.953	1.836	1.793
Speedup	1.00	2.006	3.949	6.350	6.502
Efficiency	1.00	1.003	0.987	0.794	0.406

Open_MP vs MPI vs pthreads υλοποίηση

Σύμφωνα με τα παραπάνω αποτελέσματα MPI και αυτά από την εργασία 1 και 2 παρατηρούμε πως η βέλτιστη διεπαφή για το πρόβλημα αυτό (πολλαπλασιασμό πολυωνύμων) είναι το MPI που βασίζεται στην αρχιτεκτονική κατανεμημένης μνήμης επιτρέποντας την εκτέλεση σε πολλούς κόμβους. Παρακάτω παραθέτω αρχικά τις διεπαφές από την βέλτιστη (αποδοτικότερη) στην λιγότερο βέλτιστη με βάση τους χρόνους όταν έχουν τις ίδιες παραμέτρους (ίδιο αριθμό νημάτων/διεργασιών και βαθμό πολυωνύμου):

MPI < Open_MP < pthreads

Το MPI αποδεικνύεται η βέλτιστη λύση, παρόλο που όλες οι υλοποιήσεις εκτελέστηκαν στον ίδιο κόμβο. Η καλύτερη απόδοση του MPI δεν οφείλεται στην κατανεμημένη μνήμη ή στη χρήση πολλών κόμβων, αλλά κυρίως στον τρόπο οργάνωσης και διαμοιρασμού των δεδομένων.

- **Ανεξαρτησία Υπολογισμών:** Κάθε διεργασία MPI επεξεργάζεται ανεξάρτητο τμήμα των δεδομένων και παράγει τοπικό αποτέλεσμα, χωρίς ανάγκη συγχρονισμού κατά τη διάρκεια του υπολογισμού. Ο μοναδικός συγχρονισμός πραγματοποιείται στο τέλος μέσω της MPI_Reduce.
- **Αποφυγή ανταγωνισμού σε Κοινή Μνήμη:** Σε αντίθεση με OpenMP και Pthreads, όπου όλες οι προσπελάσεις γίνονται σε κοινή μνήμη, στο MPI κάθε διεργασία χρησιμοποιεί ιδιωτικό χώρο μνήμης. Αυτό περιορίζει φαινόμενα όπως cache line invalidations.
- **Καλύτερη Τοπικότητα Δεδομένων:** Η χρήση τοπικών buffers σε κάθε διεργασία MPI οδηγεί σε καλύτερη εκμετάλλευση της cache σε σύγκριση με τις shared-memory προσεγγίσεις (OpenMP και Pthreads).

Στη συνέχεια παρόλο που το OpenMP και Pthreads βασίζονται σε κοινόχρηστη μνήμη (shared memory), το OpenMP υπερέρχει για τους εξής λόγους:

- **Thread Pooling:** Το OpenMP διατηρεί μια ομάδα έτοιμων νημάτων (pool). Έτσι, δεν σπαταλά χρόνο στη συνεχή δημιουργία και καταστροφή νημάτων.
- **Load Balancing και Scheduling:** Οι μηχανισμοί αυτόματου χρονοπρογραμματισμού του OpenMP (π.χ. dynamic scheduling) κατανέμουν τις επαναλήψεις πυρήνες πιο ομοιόμορφα, εξασφαλίζοντας ότι κανένας πυρήνας δεν μένει αδρανής ενώ άλλοι υπερφορτώνονται (workstealing).
- **Καλύτερη Διαχείριση Τοπικότητας Μνήμης:** Το OpenMP, μέσω του δομημένου μοντέλου παραλληλισμού (parallel for), διευκολύνει την πρόσβαση των νημάτων σε διαδοχικά τμήματα δεδομένων, βελτιώνοντας την τοπικότητα της cache. Αντίθετα τα Pthreads, η λανθασμένη κατανομή εργασίας μπορεί ευκολότερα να οδηγήσει σε φαινόμενα false sharing.

Τέλος τα Pthreads αποτελούν μια διεπαφή χαμηλού επιπέδου. Παρόλο που προσφέρουν απόλυτο έλεγχο στον προγραμματιστή, η χειροκίνητη διαχείριση των νημάτων οδηγεί σε:

- **Μεγάλο κόστος διαχείρισης νημάτων** Απαιτείται αρκετός χρόνος για τη δημιουργία και καταστροφή των νημάτων.
- **Αυξημένη Πολυπλοκότητα Κώδικα:** Ο προγραμματιστής είναι υπεύθυνος για τον συγχρονισμό, τη διανομή εργασίας και τη διαχείριση κοινών δεδομένων, επιβαρύνοντας τα τελικά αποτελέσματα και τον τελικό συνολικό χρόνο.

3 3.2

Πρόβλημα και Υλοποίηση

Στην άσκηση 3.2 ζητήθηκε να κατασκευάσουμε με τυχαίο τρόπο το διάνυσμα και έναν αραιό πίνακα και να τα πολλαπλασιάσουμε χρησιμοποιώντας την πυκνή και την CSR αναπαράσταση του πίνακα. Επιπλέον, για να συγκρίνουμε τις επιδόσεις ο πολλαπλασιασμός γίνεται σειριακά αλλά και παράλληλα.

Πιο συγκεκριμένα, η CSR αναπαράσταση του πίνακα κατασκευάζεται σειριακά και κάνουμε τον πολλαπλασιασμό σειριακά και με τις δυο αναπαραστάσεις του πίνακα. Στη συνέχεια στέλνουμε μέσω `MPI_Bcast()` το μέγεθος του πίνακα, τον αριθμό των επαναλήψεων και το διάνυσμα στις άλλες διεργασίες. Για τον παράλληλο πολλαπλασιασμό με CSR αναπαράσταση τα δεδομένα τα στέλνουμε με `MPI_Scatterv()`, έτσι ώστε να επιτευχθεί ισομερής κατανομή των γραμμών του πίνακα στις διεργασίες. Αυτό απαιτεί και την κατάλληλη προ-επεξεργασία από τη διεργασία 0 (δίκαιος διαχωρισμός). Η κάθε διεργασία υπολογίζει το κομμάτι που της αναλογεί και στέλνουν μέσω `MPI_allgatherv()` το τοπικό αποτέλεσμα και συλλέγουν το τελικό ολόκληρο. Ο λόγος που χρησιμοποιήθηκε το `MPI_allgatherv()` είναι ότι όλες οι διεργασίες θα χρειαστούν το τελικό αποτέλεσμα στην επόμενη επανάληψη. Στην πυκνή αναπαράσταση η διεργασία 0 μοιράζει τον πίνακα με `MPI_Scatter()`. Όλες οι διεργασίες υπολογίζουν τον κομμάτι που τους αναλογεί και στο τέλος με `MPI_allgather()` παίρνουν το τελικό αποτέλεσμα. Στην περίπτωση που περισσεύουν κάποιες γραμμές του πίνακα η διεργασία 0 αναλαμβάνει να κάνει τον πολλαπλασιασμό και μετά στέλνει το αποτέλεσμα σε όλες τις υπόλοιπες διεργασίες μέσω `MPI_Bcast()`.

Αποτελέσματα

Σημείωση: Για όλα τα παραδείγματα τρέξαμε και για παραπάνω από 8 διεργασίες αλλά όπως φαίνεται και στη πορεία αυτό δε επιταχύνει την εκτέλεση του προγράμματος αφού ο χρόνος των υπολογισμών είναι μικρότερος από τον χρόνο επικοινωνίας μεταξύ των διεργασιών. Επιπλέον, με τον όρο "Χρόνος διαμοιρασμού data" αναφερόμαστε στον χρόνο που απαιτείται από τη διεργασία 0 για να αποστείλει τα δεδομένα στις υπόλοιπες διεργασίες.

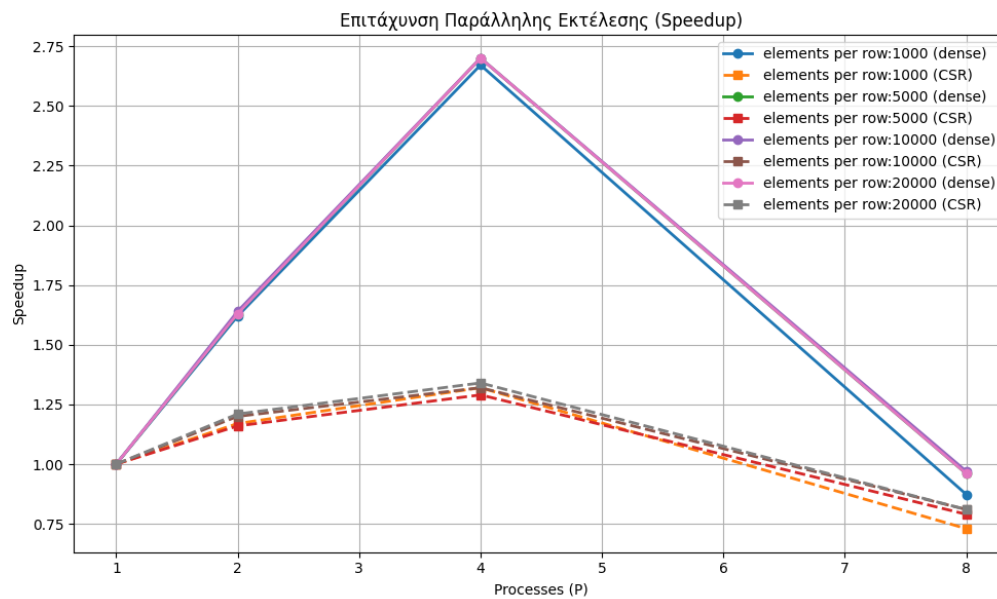
Μεταβολή στο μέγεθος του πίνακα: Για να συγκρίνουμε τον αριθμό των στοιχείων ανά γραμμή κρατήσαμε σταθερό τον αριθμό επαναλήψεων και το ποσοστό μηδενικών στοιχείων, 10 και 50% αντίστοιχα.

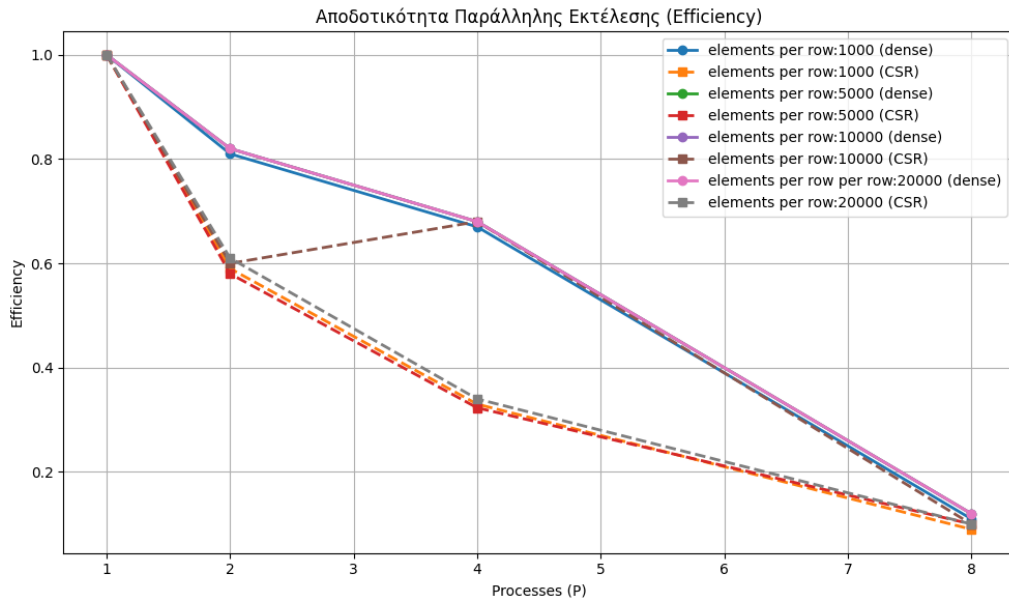
Μέγεθος πίνακα	1000							
Αριθμός διεργασιών	1		2		4		8	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος κατασκευής CSR	-	0.0177	-	0.0177	-	0.0177	-	0.0177
Χρόνος διαμοιρασμού data	-	-	0.0021	0.0021	0.0022	0.0019	0.0204	0.0211
Χρόνος υπολογισμού	0.0219	0.0143	0.0115	0.0075	0.006	0.004	0.0047	0.0044
Συνολικός Χρόνος	0.0219	0.032	0.0135	0.0273	0.0082	0.0242	0.0251	0.0439
Speedup	1.00	1.00	1.62	1.17	2.671	1.32	0.873	0.73
Efficiency	1.00	1.00	0.81	0.59	0.67	0.33	0.11	0.09

Μέγεθος πίνακα	5000							
Αριθμός διεργασιών	1		2		4		8	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος κατασκευής CSR	-	0.448	-	0.448	-	0.448	-	0.448
Χρόνος διαμοιρασμού data	-	-	0.0534	0.054	0.0558	0.054	0.503	0.491
Χρόνος υπολογισμού	0.558	0.3308	0.288	0.189	0.152	0.1	0.078	0.053
Συνολικός Χρόνος	0.558	0.786	0.342	0.68	0.207	0.61	0.581	1.00
Speedup	1.00	1.00	1.63	1.16	2.7	1.29	0.96	0.79
Efficiency	1.00	1.00	0.82	0.58	0.68	0.323	0.12	0.10

Μέγεθος πίνακα	10000							
Αριθμός διεργασιών	1		2		4		8	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος κατασκευής CSR	-	1.79	-	1.79	-	1.79	-	1.79
Χρόνος διαμοιρασμού data	-	-	0.213	0.214	0.224	0.215	2.011	1.985
Χρόνος υπολογισμού	2.237	1.46	1.154	0.753	0.606	0.401	0.306	0.203
Συνολικός Χρόνος	2.237	3.25	1.367	2.706	0.829	2.458	2.317	4.013
Speedup	1.00	1.00	1.64	1.2	2.70	1.32	0.97	0.81
Efficiency	1.00	1.00	0.82	0.6	0.68	0.68	0.12	0.10

Μέγεθος πίνακα	20000							
Αριθμός διεργασιών	1		2		4		8	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος κατασκευής CSR	-	7.26	-	7.26	-	7.26	-	7.26
Χρόνος διαμοιρασμού data	-	-	0.855	0.85	0.887	0.861	8.083	8.048
Χρόνος υπολογισμού	8.934	5.83	4.638	3.024	2.425	1.591	1.221	0.813
Συνολικός Χρόνος	8.934	13.09	5.493	10.82	3.312	9.763	9.303	16.17
Speedup	1.00	1.00	1.63	1.21	2.70	1.34	0.96	0.81
Efficiency	1.00	1.00	0.82	0.61	0.68	0.34	0.12	0.10





Από τα παραπάνω μπορούμε να συμπεράνουμε πως στο συγκεκριμένο πρόβλημα δεν "συμφέρει" να χρησιμοποιήσουμε παραλληλοποίηση μέσω MPI ούτε για πίνακες με λίγα στοιχεία ανά γραμμή ούτε για αυτούς με πολλά στοιχεία. Αυτό συμβαίνει καθώς στην υλοποίηση με MPI η διεργασία 0 καλείται να σπάσει και να στείλει τον πίνακα σε τμήματα στις υπόλοιπες διεργασίες. Με βάση τα στοιχεία ανά γραμμή που δοκιμάστηκαν συμπεραίνουμε πως όταν χρησιμοποιούμε έως 4 διεργασίες υπάρχει βελτίωση χρόνου με χαμηλή απόδοση όμως. Αλλά όταν δοκιμάζουμε 8+ η υλοποίηση με παραλληλία καταλήγει να είναι πιο αργή από την σειριακή υλοποίηση. Επιπλέον, βλέπουμε πως ο πολλαπλασιασμός χρησιμοποιώντας την πυκνή αναπαράσταση του πίνακα παίρνει λιγότερο χρόνο σε σχέση με την υλοποίηση που χρησιμοποιεί την CSR αναπαράσταση. Αυτό συμβαίνει καθώς στην CSR αναπαράσταση απαιτείται έξτρα χρόνος για τη δημιουργία του CSR πίνακα.

Μεταβολή στο ποσοστό των μηδενικών στοιχείων

Για να συγκρίνουμε το ποσοστό μηδενικών στοιχείων του πίνακα κρατήσαμε σταθερό τον αριθμό των στοιχείων ανά γραμμή και τον αριθμό των επαναλήψεων, 10000 και 10 αντίστοιχα.

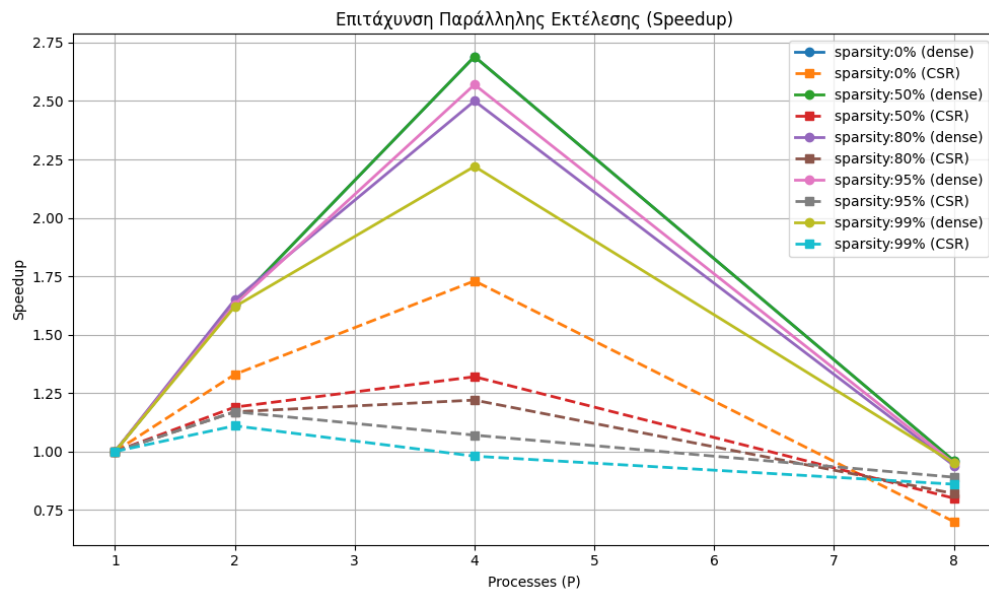
Ποσοστό μηδενικών στοιχείων	0%							
Αριθμός διεργασιών	1		2		4		8	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος κατασκευής CSR	-	0.98	-	0.98	-	0.98	-	0.98
Χρόνος διαμοιρασμού data	-	-	0.213	0.431	0.224	0.431	2.012	4.015
Χρόνος υπολογισμού	2.231	2.886	1.154	1.493	0.607	0.788	0.306	0.474
Συνολικός Χρόνος (sec)	2.231	3.866	1.37	2.90	0.83	2.24	2.32	5.52
Speedup	1.00	1.00	1.63	1.33	2.69	1.73	0.96	0.7
Efficiency	1.00	1.00	0.82	0.67	0.67	0.43	0.12	0.09

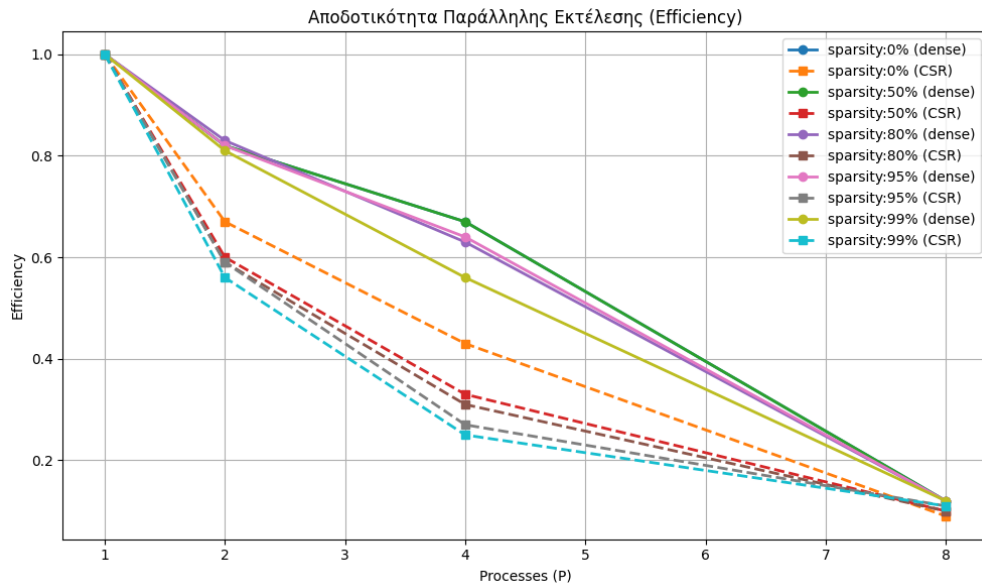
Ποσοστό μηδενικών στοιχείων	50%							
Αριθμός διεργασιών	1		2		4		8	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος κατασκευής CSR	-	1.76	-	1.76	-	1.76	-	1.76
Χρόνος διαμοιρασμού data	-	-	0.214	0.214	0.222	0.215	2.017	1.987
Χρόνος υπολογισμού	2.233	1.457	1.154	0.752	0.606	0.397	0.307	0.203
Συνολικός Χρόνος (sec)	2.233	3.22	1.368	2.706	0.828	2.438	2.324	4.042
Speedup	1.00	1.00	1.63	1.19	2.69	1.32	0.96	0.8
Efficiency	1.00	1.00	0.82	0.6	0.67	0.33	0.12	0.1

Ποσοστό μηδενικών στοιχείων	80%							
Αριθμός διεργασιών	1		2		4		8	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος κατασκευής CSR	-	0.988	-	0.988	-	0.988	-	0.988
Χρόνος διαμοιρασμού data	-	-	0.212	0.086	0.317	0.097	2.010	0.802
Χρόνος υπολογισμού	2.312	0.614	1.187	0.316	0.607	0.206	0.446	0.095
Συνολικός Χρόνος (sec)	2.312	1.602	1.399	1.364	0.923	1.308	2.457	1.949
Speedup	1.00	1.00	1.65	1.17	2.5	1.22	0.94	0.82
Efficiency	1.00	1.00	0.83	0.59	0.63	0.31	0.12	0.1

Ποσοστό μηδενικών στοιχείων	95%							
Αριθμός διεργασιών	1		2		4		8	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος κατασκευής CSR	-	0.603	-	0.603	-	0.603	-	0.603
Χρόνος διαμοιρασμού data	-	-	0.221	0.021	0.277	0.022	2.066	0.199
Χρόνος υπολογισμού	2.276	0.18	1.176	0.082	0.607	0.070	0.327	0.042
Συνολικός Χρόνος (sec)	2.276	0.783	1.397	0.668	0.884	0.734	2.393	0.88
Speedup	1.00	1.00	1.63	1.17	2.57	1.07	0.95	0.89
Efficiency	1.00	1.00	0.82	0.59	0.64	0.27	0.12	0.11

Ποσοστό μηδενικών στοιχείων	99%							
Αριθμός διεργασιών	1		2		4		8	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος κατασκευής CSR	-	0.53	-	0.53	-	0.53	-	0.53
Χρόνος διαμοιρασμού data	-	-	0.218	0.00445	0.306	0.00404	2.053	0.0446
Χρόνος υπολογισμού	2.273	0.0341	1.181	0.0169	0.720	0.00907	0.349	0.0228
Συνολικός Χρόνος (sec)	2.273	0.564	1.399	0.509	1.026	0.573	2.402	0.658
Speedup	1.00	1.00	1.62	1.11	2.22	0.98	0.95	0.86
Efficiency	1.00	1.00	0.81	0.56	0.56	0.25	0.12	0.11





Από τα παραπάνω μπορούμε να συμπεράνουμε πως ανεξαρτήτως το ποσοστό των μηδενικών στοιχείων για να είναι η παραλληλοποίηση αποδοτική θα πρέπει να χρησιμοποιήσουμε μέχρι 4 κόμβους. Διαφορετικά ο χρόνος, που απαιτείται από τη διεργασία 0 για τη διάσπαση του πίνακα σε όλες τις διεργασίες αλλά και η συλλογή του αποτελέσματος μέσω `MPI_allgather()` είναι πολύ μεγαλύτερος από τον χρόνο που απαιτείται για να πραγματοποιηθεί ο πολλαπλασιασμός. Επιπλέον, παρατηρούμε πως ο πολλαπλασιασμός με CSR αναπαράσταση είναι ταχύτερος για αραιό πίνακα σε σχέση με την πυκνή αναπαράσταση. Πιο συγκεκριμένα αυτό συμβαίνει όταν το ποσοστό των μηδενικών στοιχείων είναι μεγαλύτερο του 80%.

Μεταβολή στον αριθμό των επαναλήψεων

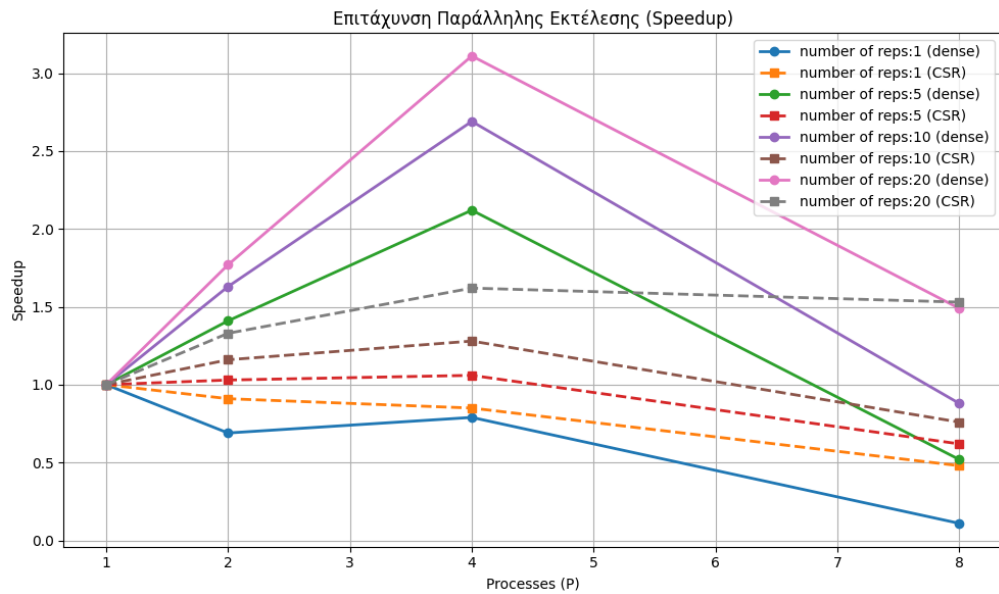
Για να συγκρίνουμε τον αριθμό των επαναλήψεων ανά γραμμή κρατήσαμε σταθερό τον αριθμό των στοιχείων ανά γραμμή και το ποσοστό μηδενικών στοιχείων, 10000 και 50% αντίστοιχα.

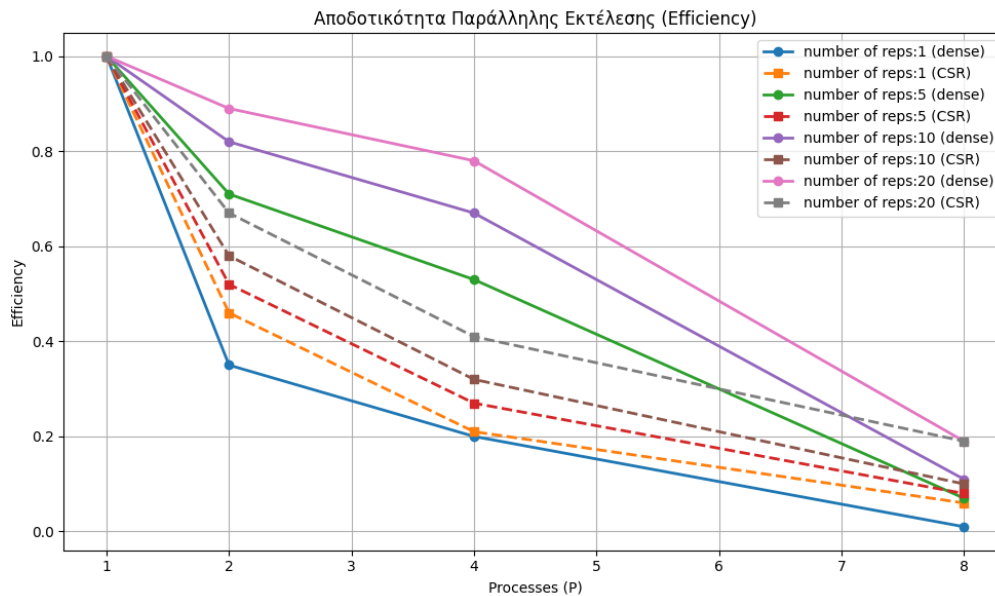
Αριθμός επαναλήψεων	1							
Αριθμός διεργασιών	1		2		4		8	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος κατασκευής CSR	-	1.722	-	1.767	-	1.929	-	1.869
Χρόνος διαμοιρασμού data	-	-	0.212	0.216	0.224	0.229	2.003	2.011
Χρόνος υπολογισμού	0.224	0.150	0.116	0.0758	0.0607	0.0399	0.0313	0.0214
Συνολικός Χρόνος (sec)	0.224	1.872	0.327	2.059	0.284	2.198	2.034	3.902
Speedup	1.00	1.00	0.69	0.91	0.79	0.85	0.11	0.48
Efficiency	1.00	1.00	0.35	0.46	0.2	0.21	0.01	0.06

Αριθμός επαναλήψεων	5							
Αριθμός διεργασιών	1		2		4		8	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος κατασκευής CSR	-	1.682	-	1.739	-	1.845	-	1.823
Χρόνος διαμοιρασμού data	-	-	0.211	0.212	0.222	0.215	2.008	1.989
Χρόνος υπολογισμού	1.116	0.728	0.578	0.380	0.303	0.199	0.154	0.102
Συνολικός Χρόνος (sec)	1.116	2.41	0.789	2.331	0.525	2.259	2.162	3.914
Speedup	1.00	1.00	1.41	1.03	2.12	1.06	0.52	0.62
Efficiency	1.00	1.00	0.71	0.52	0.53	0.27	0.07	0.08

Αριθμός επαναλήψεων	10							
Αριθμός διεργασιών	1		2		4		8	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος κατασκευής CSR	-	1.684	-	1.738	-	1.832	-	1.825
Χρόνος διαμοιρασμού data	-	-	0.214	0.213	0.223	0.214	2.043	1.993
Χρόνος υπολογισμού	2.233	1.459	1.158	0.755	0.606	0.401	0.488	0.337
Συνολικός Χρόνος (sec)	2.233	3.143	1.372	2.706	0.829	2.447	2.531	4.155
Speedup	1.00	1.00	1.63	1.16	2.69	1.28	0.88	0.76
Efficiency	1.00	1.00	0.82	0.58	0.67	0.32	0.11	0.1

Αριθμός επαναλήψεων	20							
Αριθμός διεργασιών	1		2		4		8	
Αναπαράσταση	dense	CSR	dense	CSR	dense	CSR	dense	CSR
Χρόνος κατασκευής CSR	-	1.683	-	1.738	-	1.832	-	1.835
Χρόνος διαμοιρασμού data	-	-	0.212	0.214	0.222	0.215	2.029	1.994
Χρόνος υπολογισμού	4.462	2.914	2.310	1.510	1.213	0.795	0.966	0.665
Συνολικός Χρόνος (sec)	4.462	4.597	2.522	3.462	1.435	2.842	2.995	4.495
Speedup	1.00	1.00	1.77	1.33	3.11	1.62	1.49	1.53
Efficiency	1.00	1.00	0.89	0.67	0.78	0.41	0.19	0.19





Από τα παραπάνω αποτελέσματα προκύπτει ότι, για όλες τις τιμές επαναλήψεων, η καλύτερη κλιμάκωση επιτυγχάνεται όταν γίνεται χρήση έως και 4 διεργασιών. Για μεγαλύτερο αριθμό διεργασιών (π.χ. 8), το όφελος περιορίζεται, καθώς ο χρόνος επικοινωνίας (διαμοιρασμός δεδομένων και συλλογή αποτελεσμάτων με `MPI_Allgather()`) παραμένει σημαντικός και συχνά υπερκαλύπτει τη μείωση του υπολογιστικού χρόνου. Επιπλέον, καθώς αυξάνεται ο αριθμός επαναλήψεων, το σταθερό κόστος (κατασκευή CSR και επικοινωνία) αποσβένεται, με αποτέλεσμα η CSR αναπαράσταση να γίνεται ολοένα και πιο ανταγωνιστική και να "ζυγώνει" τους χρόνους της πυκνής αναπαράστασης.

Τελικό συμπέρασμα

Τέλος, από τις δοκιμές βγαίνουν τα εξής συμπεράσματα:

- Ο πολλαπλασιασμός αραιού πίνακα με διάνυσμα χρησιμοποιώντας την CSR αναπαράσταση είναι πιο γρήγορος όταν το ποσοστό των μηδενικών στοιχείων είναι αρκετά μεγάλο ξεπερνάει το ποσοστό του 80% ή όταν ο αριθμός των επαναλήψεων είναι μεγάλος (περισσότερες από 25-30 επαναλήψεις). Σε όλες τις υπόλοιπες περιπτώσεις ο πολλαπλασιασμός με τη χρήση της πυκνής αναπαράστασης είναι πιο γρήγορος.
- Η παραλληλοποίηση μέσω MPI στο συγκεκριμένο πρόβλημα επιταχύνει τον χρόνο εκτέλεσης μόνο όταν χρησιμοποιούμε έως 4 διεργασίες. Διαφορετικά, ο χρόνος αυξάνεται σε μεγάλο βαθμό αφού ο χρόνος εκτέλεσης των υπολογισμών είναι αρκετά μικρότερος σε σχέση με τον χρόνο που απαιτείται για να επικοινωνήσουν οι διεργασίες μεταξύ τους.
- Συγκρίνοντας τις επιδόσεις της παραλληλοποίησης με MPI και OpenMP συμπεραίνουμε πως η υλοποίηση με OpenMP είναι πιο γρήγορη αφού η επιτάχυνση είναι αρκετά μεγαλύτερη (4-5 φορές). Αυτό συμβαίνει για μικρό φόρτο εργασίας όπως τώρα.